CAHIER DES CHARGES

Démineur

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

INTRODUCTION

Nom du projet : Démineur
N° du Projet : Programmation Impérative C++
Nom d'UE : Fondamentaux
N° d'UE : 1
Date : 07/10/2024 – 25/10/2024
Classe : 1ère année

Modalités

Projet solo.

Contexte et Description du Projet

L'objectif de ce projet est de réaliser **un démineur en invite de commande** en **C++** sans **aucune librairie tierce**.

Dans le démineur le joueur fait face à une grille avec des **cases masquées** et doit tenter de révéler toutes les cases hormis celles abritant **un nombre défini de bombes**. La partie se termine quand seule les bombes n'ont pas été révélé.

Pour parvenir à découvrir les bombes, le joueur révèlera des cases qui sont des **chiffres** indiquant **exactement** le nombre de **bombes adjacentes**.

Afin de s'aider dans cette tâche ardue, le joueur pourra mettre des **« flags »** sur des cases quand il suspecte qu'il y a une bombe cachée.

Le joueur pourra indiquer une taille de grille cohérente et un niveau de difficulté pour générer une partie et afficher la grille dans le terminal.

L'objectif est donc de créer ce jeu complet, dans le terminal, avec un écran de fin en cas de défaite ou de victoire.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

Le joueur pourra indiquer des coordonnées suivies du mot clef « clic » ou « flag » en fonction de ce qu'il souhaite faire, « découvrir » ou « marquer » la zone.

À chaque clic :

- Si le joueur vise une case n'ayant aucune bombe adjacente, le jeu devra automatiquement révéler toutes les cases voisines jusqu'à qu'elles atteignent des cases adjacentes aux bombes.
- Si le joueur vise un chiffre, seul le chiffre sera révélé.
- Si le joueur vise une bombe, toutes les bombes sont révélées et la partie est terminée (perdu).

Les flags sont justes des éléments d'information pour le joueur.

Specs

Vous devez implémenter dans votre application :

- Gameplay
  - Génération de grille
    - Le joueur pourra indiquer une taille de grille cohérente et un niveau de difficulté pour la générer (approche ouverte à définir et à présenter le jour de la soutenance)
    - La grille sera tout d'abord générée avec des cases masquées. Les cases masquées cacheront :
      - Soit des bombes
      - Soit des valeurs
      - Soit des cases vides.
    - La grille sera affichée dans le terminal et les cases seront identifiables via des caractères en fonction de leurs natures
  - Commande joueur
    - Le joueur indiquera les coordonnées de la case avec laquelle il souhaite interagir suivie de la commande
      - « clic » pour cliquer et découvrir la case
      - « flag » pour marquer la zone s'il suspecte une bombe cachée
  - Mécanisme de révélation
    - À chaque clic

            - Si le joueur vise une case n'ayant aucune bombe adjacente, il devra découvrir les cases voisines qui elles, ont des cases adjacentes aux bombes
            - Si le joueur vise un chiffre, seul le chiffre sera révélé
            - Si le joueur vise une bombe, toutes les bombes sont révélées et la partie est terminée
    - Fin de partie
        - Si le joueur clique sur une bombe
        - Si le joueur a révélé toutes les cases qui ne sont pas des bombes

- Outils
    - Les étudiants ne devront pas utiliser les conteneurs de la librairie standard et devront implémenter leurs propres conteneurs.
    - Ils devront versionner leur code en utilisant git afin d'avoir une phase de développement saine.

<u>Pour les plus avancés uniquement</u> (ou pour aller plus loin)

- **Save and load :** ajout des commandes « save » pour sauvegarder la partie dans un fichier et « load » pour charger une partie préalablement sauvegardée.
- **Triche ++ :** Mettre en place un mécanisme de save and load qui ne soit pas facilement manipulable par le joueur qui souhaite tricher.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

OBJECTIFS PÉDAGOGIQUES ET PROFESSIONNELS DU PROJET

A l'issue de ce projet, les étudiants seront capables de :

Savoirs

- **Modéliser et adapter** des algorithmes mathématiques en C++ pour répondre à un besoin défini.
- **Décomposer** un problème en différentes parties fonctionnelles afin d'atteindre des objectifs intermédiaires.
- **D'analyser** leur code via le débugger pour identifier une potentielle erreur.
- **Comprendre** la structure interne des conteneurs principaux de la librairie standard du C++.
- **Expliquer** le fonctionnement de l'algorithme « flood fill » qui est un des plus vieux algorithmes du jeu vidéo présent dans tous les logiciels de dessin et de modification de photographies.
- **Décrire** ce que sont des pointeurs et des références et d'expliquer comment cela se passe au niveau de la mémoire.
- **D'identifier et développer** tous les algorithmes de tri populaires.
- **Composer** avec différents types de données.
- **Manipuler** l'IDE le plus utilisé dans l'industrie.

Savoir-être

- **Construire** un environnement organisé pour le développement.
- **Découper** des tâches en sous-tâches.

Savoir-faire / Compétences

- **Programmer** en utilisant les bases du C++.

RESSOURCES

1. **A installer/prérequis techniques** :
    - Visual Studio 2022
    - Git
    - Helix (p4merge)

INTERVENANTS

Nom : OUKRAT Rémi
Titre : Architecte logiciel
Email : remi.oukrat@gmail.com
LinkedIn : www.linkedin.com/in/remi-o

BIOGRAPHIE

Depuis mon plus jeune âge, je suis passionné par les jeux vidéo. J'ai fait des études en mathématiques pures et en physique. Ma passion m'a guidé vers une école d'ingénieur pour me permettre d'apprendre le C++, qui est le langage du secteur du jeu vidéo. Cela m'a permis, de combiner mes connaissances théoriques et pratiques afin de réaliser mes objectifs : concevoir des jeux vidéo. J'ai travaillé dans différents secteurs d'activité : aéronautique, divertissement, robotique, financier, scientifique, etc... en tant que programmeur et architecte logiciel. J'ai commencé par travailler sur le développement de système d'exploitation et de composants électroniques jusqu'à la réalisation de jeux et de logiciels plus haut niveau. J'ai utilisé de nombreux moteurs de jeux populaires et j'ai travaillé sur le développement de briques logicielles dans des moteurs du secteur. J'ai également travaillé sur les librairies standard C++ utilisées par certains moteurs. J'ai architecturé de nombreux moteurs qui géraient du rendu, de la physique et divers éléments d'IA pour le gameplay. Je travaille actuellement en tant qu'architecte dans une entreprise scientifique. J'enseigne le C++, l'architecture logicielle et le développement de jeu vidéo à des Master 2 depuis 2016. Je fais aussi des formations en entreprise sur le C++ depuis 2020.
J'écris des articles techniques sur le jeu vidéo pour certains magazines et je participe à des émissions pour vulgariser des points techniques. En 2023, j'ai organisé en collaboration avec un site de jeux vidéo un gros évènement lors du Xbox Showcase, où j'ai fait venir des grands noms du secteur.
Pour faire simple, le jeu vidéo est ma passion et c'est un plaisir de la partager !

Nom : LAVIGNE Quentin
Titre : Architecte logiciel
Email : quentin.lavigne@thermofisher.com

Biographie

Autodidacte en programmation, j'ai découvert celle-ci pratiquement en même temps que l'écriture puisque âgé de 6 ans et demi ! J'ai fait mes premières armes avec GW Basic, Turbo Basic puis QuickBasic avant de passer au Visual Basic vers mes 13 ans. Finalement, vers mes 14 ans j'ai franchi le pas d'apprendre le C++ afin de pouvoir programmer des jeux vidéo et suis tombé amoureux du langage...
D'un point de vue académique, j'ai fait des études de mathématiques pures (une autre de mes passions) durant lesquelles j'étais particulièrement attiré par la topologie et la géométrie différentielle. J'ai également un vif intérêt pour la physique et la cosmologie.
Professionnellement, j'ai eu l'occasion de programmer dans des domaines très variés : le management de projet, le calcul scientifique, la visualisation 3D temps réel, toujours en C++ !
Aujourd'hui, j'ai envie de faire découvrir mes passions

ROADMAP

Description

Les dates limites fixées doivent être rigoureusement respectées afin de garantir une évaluation équitable.

Jalon	Livrables attendus	Date limite	Moyens / formats
1	**Remise du projet**	24/10/2024 à 18h00	• Repository git

2	**Soutenance**	25/10/2024 de 13h00 à 19h00	• Soutenance orale

EVALUATION

L'évaluation est conçue pour être holistique, prenant en compte non seulement le produit final, mais aussi le processus, les compétences acquises et les attitudes démontrées tout au long du projet. Les compétences sont classées en trois catégories principales : "savoirs" (connaissances théoriques), "savoir-faire" (compétences pratiques), et "savoir-être" (compétences interpersonnelles et attitudes).

Système de Notation

Savoirs (Connaissances)

- **Compréhension théorique** : Évaluée soit en amont du projet pendant la semaine théorie soit lors de la soutenance et restitution du projet. Cela permet de mesurer la compréhension des concepts fondamentaux et des connaissances liées au projet des étudiants.

Savoir-faire (Compétences)

- **Compétences techniques et application** : Évaluées à travers la soumission finale du projet. Cela inclut la qualité, la fonctionnalité, et la précision technique du travail produit.
- **Gestion de projet** : Évaluée en fonction de l'organisation, du respect des échéances, et de l'utilisation efficace des ressources. Cela peut être évalué à travers la documentation du projet et les journaux de processus.

Savoir-être (Attitudes/Compétences interpersonnelles)

- **Travail d'équipe et collaboration** : Évalués à travers des évaluations par les pairs et les membres du groupe. Les critères incluent la communication, la coopération, et la contribution aux tâches du groupe.
- **Autonomie et initiative** : Évaluées en fonction des contributions individuelles, de la capacité à travailler de manière autonome, et de la résolution proactive des problèmes.
- **Soutenance** : Évaluées lors de la présentation finale du projet. Les critères incluent la clarté, la cohérence, et la capacité à articuler et défendre les résultats du projet.

GRILLE D'ÉVALUATION

Évaluation du produit fini	Note
Respect du cahier des charges et fonctionnalités implémentées	10
Interface utilisateur	5
Gestion des ressources	5
Total	**20**

Évaluation de la soutenance	Note
Aisance à l'oral	2
Justification des choix	4
Présentation de l'architecture	4
Respect du temps de parole	2
Organisation du travail	3
Améliorations proposées et problèmes rencontrés	3
Qualité des slides	2
Total	**20**

Questions/réponses	Note
Total	**20**

Notation sur le code	Note
Architecture	10
Algorithmique	10
Build system/git	10
Qualité du C++	10
Total	**40**

La note finale est la note sur 100 ramenée sur 20.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

Modalités de la soutenance :

- 30 minutes réparties de la manière suivante
    - 10 minutes de présentation PowerPoint ou similaire
    - 5 minutes de test du jeu
    - 10 minutes de questions
    - 5 minutes d'évaluation de la soutenance

EXAMEN INDIVIDUEL DE FIN DE PÉRIODE
Examen de 2h sur ordinateur comprenant :

- QCM
- Des exercices de C++ et d'algorithmique